Weekly Report

Ali Alhulaimi
alialhulaimi2005@gmail.com

Abstract

This week the work moved from making the model's output printable after the fact to improving the model itself. I first extended the post-processing side: I tried many post-processing libraries and methods to improve quality and printability, adding different options for post-processing, and found manifold3d to be the best for 3D printing, which I added while keeping the original trimesh pipeline in place. I also did further research on the best way to decrease non-manifold edges of the models, and read the TriFlow paper on generating artist-like 3D models. I then tried to integrate DreamDPO concepts into the pipeline, without success: I read the DreamDPO work for diffusion models, but simulated the preference discriminator as a score rather than a VLM, and the results showed no gains - it always chose the reference path for the flow model. That, together with my lack of expertise in this area of the theory, led me to work instead on fine-tuning the SLat flow model using LoRA methods. I tried this on a small sample size of 300 curated 3D models from Thingi10K, and initial training showed gains, with significantly less non-manifold edges for raw results and without loss in detail. I am continuing to collect more latent data for training. This report covers that work, together with the deployment and validation-layer work and the background material on the TRELLIS pipeline and on flow and diffusion models reviewed over the previous weeks.

Date: August 2026
Explainer: https://claude.ai/public/artifacts/c52594ab-8c5c-404c-8999-68e62b4054ee
Service: https://scifablabs-mac-mini.tailfc1a5e.ts.net/

Contents

1 Introduction

1.1 Overview

This report covers the topics I have worked through over the past few weeks while preparing for the TRELLIS project: the TRELLIS pipeline itself and the flow/diffusion theory that TRELLIS is built on, along with my work running the model, post-processing its output, and - this week - putting it behind a web front end so that it can be used in the FabLab.

1.2 Report Roadmap

Section 2 lists the sources I studied over these past weeks. Section 3 summarizes the topics themselves, one subsection per topic. Section 4 covers my initial results, including the validation layer I built to make the model's output printable, together with the printed output and photos. Section 5 covers the deployment of the pipeline as a web service for FabLab use. Sections 6 to 8 cover this week's work: extending the post-processing layer, an attempt at preference optimization, and fine-tuning the SLat flow model with LoRA. Section 9 is a short discussion, Section 10 covers limitations and next steps, and Section 11 concludes.

2 Related Work

The material I studied over the past few weeks comes from the MIT 6.S184 lecture notes on flow and diffusion models, and the TRELLIS and TRELLIS-2 papers on structured 3D latents, along with the DINOv3 paper referenced as an image feature extractor. This week I also read the TriFlow paper on generating artist-like 3D models and the DreamDPO work on preference optimization for diffusion models, and used the Thingi10K dataset for fine-tuning. Full citations are in the References section.

3 Topics Covered Over the Past Few Weeks

3.1 TRELLIS Pipeline Overview

I reviewed how the TRELLIS pipeline works end to end. Training data comes from three datasets. A 2D vision model (DinoV2) extracts visual features, while a voxelize step extracts spatial features. Both sets of features get a positional embedding and are passed into a VAE encoder, which produces a structured latent (SLat). The SLat then gets another positional embedding and is passed into a VAE decoder to reconstruct the output. This encoder-decoder setup is a variational autoencoder.

TRELLIS uses rectified flow transformers instead of regular transformers. Regular transformers predict the next token, but rectified flow transformers predict a continuous value instead, such as geometry, color, or texture. TRELLIS also uses time-adaptive normalization, and its tokens are made up of voxels and image patches. An interactive explainer of how this pipeline works is available online [5]; Figure 1 shows its stage-by-stage overview.

Figure 1 The five stages of the TRELLIS-2 image-to-3D pipeline, from DINOv3 image features through to the final PBR mesh, with the tensor shape handed between stages. Captured from the interactive explainer [5].

3.2 Foundations of Flow and Diffusion Models

I reviewed the general theory behind flow and diffusion models from the MIT lecture notes. Generation is

framed as representing objects as vectors, for example images, videos, or molecular structures, each living in their own vector space. The data distribution is the distribution of objects we want to generate, and generating an object means sampling from it. The initial distribution is usually a standard Gaussian.

A flow model depends on an ordinary differential equation (ODE). A trajectory evolves over time according to a vector field, which assigns a direction to every point in time:

$$d/dt\ X\,t\ =\ u\,t\,(X\,t).$$

In practice this ODE is simulated step by step using the Euler method, starting from an initial point and repeatedly taking small steps in the direction of the vector field.

3.3 Flow Matching

Flow matching is how a flow model is actually trained. The goal is to learn a vector field u t theta, using a neural network, so that simulating its ODE moves samples from the initial (noise) distribution to the data distribution. Since the true target vector field can't be computed directly, training instead uses the conditional flow matching loss,

$$L\ CFM\ (\ theta\ )\ =\ E\ [\ ||\ u\ t\ theta\ (\ x\ )\ -\ u\ t\ target\ (\ x\ |\ z\ )\ ||\ \hat{}2\ ],$$

which turns out to have the same gradient as the loss we actually want to minimize. Training samples a data point, a random time, and some noise, combines them into a single point along the path between noise and data, and updates the network to match the target direction at that point. The interactive explainer (Figure 2) visualizes this as draining a pool of noise into structure.

Figure 2 Flow matching shown as a "pool" analogy in the interactive explainer [5]: a learned velocity field carries particles from a pool of Gaussian noise (right) toward the structured voxel occupancy of the object (left, the T-shape), integrated with Euler steps at inference. The live version is interactive; a static view is shown here.

3.4 Classifier and Classifier-Free Guidance

I then reviewed guidance, which is how prompts get incorporated into generation. An unguided model just generates an image; a guided model generates, for example, "an image of a cat" given a prompt. Classifier

guidance combines the unguided vector field with the gradient of a separately trained classifier. Classifier-free guidance (CFG) avoids training that separate classifier by instead combining a guided and an unguided vector field directly:

$$u\tilde{}_t^w(x \mid y) = w\, u_t^{\text{target}}(x \mid y) + (1 - w)\, u_t^{\text{target}}(x), \quad w >= 1,$$

where w controls how strongly the prompt is enforced, and the unguided term is just the same network run with an empty prompt. Bayes' theorem provides the background for this derivation.

3.5 Latent Spaces and Variational Autoencoders

I also looked at latent spaces and variational autoencoders (VAEs), which is how flow and diffusion models are made to work on compressed representations instead of raw data. A VAE encodes data into a distribution over a smaller latent space and decodes it back, trained with a loss that balances two goals: reconstructing the original data well, and keeping the latent distribution close to a standard Gaussian (measured with the KL divergence). This also includes the reparameterization trick, which makes it possible to backpropagate through the random sampling step, and the full beta-VAE training algorithm.

3.6 Latent Diffusion Models and Transformer Architecture

With VAEs covered, I looked at how latent diffusion models (LDMs) put everything together: take all the data, encode it into latents with a VAE, train a diffusion/flow model on this smaller latent dataset, and decode samples back to the original, uncompressed format after sampling. A concrete example of the size reduction this gives is an image going from shape [3, 256, 256] down to a latent of shape [4, 32, 32].

I also reviewed the neural network architecture used to implement the guided vector field: time is encoded with a sinusoidal embedding, the prompt is encoded with a pretrained language or image model (CLIP for text, or DINOv3 for image prompts, as used in 3D generation), and the latent image is split into patches. These three embeddings are combined inside a Diffusion Transformer (DiT), which uses self-attention over the image and cross-attention to bring in the prompt, with a time-adaptive layer norm to bring in the time information. As a case study, I looked at Stable Diffusion 3: a flow matching model with a straight-line scheduler, classifier-free guidance, and about 8 billion parameters, trained on the LAION dataset.

4 Results

Alongside the background reading above, I have also been running the TRELLIS model over the past few weeks. The model's raw output was not directly usable, so I built a validation layer that post-processes the model's output to make it printable. Photos of the printed output are shown below.

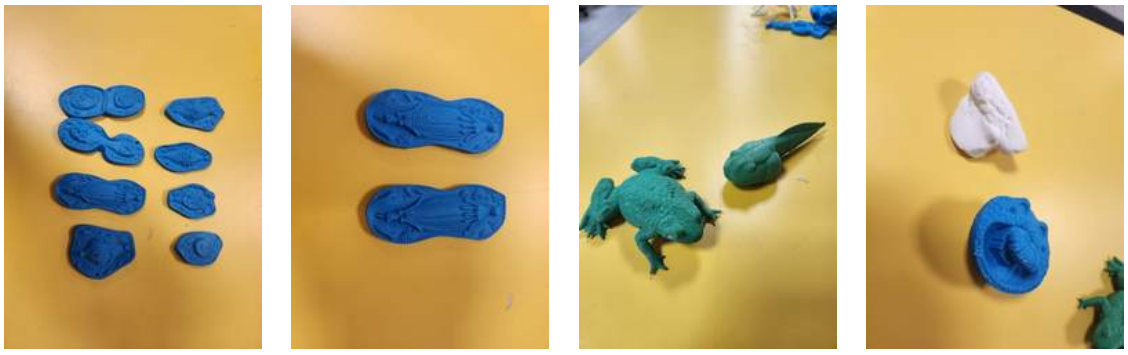

Figure 3 Physical 3D prints of TRELLIS-generated models after post-processing through the validation layer. The validation layer repairs the model's raw output into a watertight, printable mesh.

5 Web Deployment for FabLab Use

5.1 Motivation

The main work this week was moving the model off my local command line and onto a web service so that it can actually be used in the FabLab. Running TRELLIS directly requires a GPU environment, the model weights, and a manual call to the validation layer described in Section 4. None of that is reasonable to ask of a FabLab user who simply wants a printable model from a photograph, so I wrapped the whole pipeline behind a web front end, SOLIDIFY, served from the lab machine.

5.2 SOLIDIFY: Photo In, Mesh Out

The interface follows the pipeline in one direction. The user supplies a single 2D photograph, the server runs the TRELLIS image-to-3D pipeline on it, the raw mesh is passed through the validation layer, and what comes back is a watertight STL that can be sent straight to the slicer. The intermediate representations discussed in Section 3 - image features, the sparse structure, the structured latent - are all hidden from the user; from the outside the site takes a photo in and gives matter out.

Figure 4 shows a field test through the deployed service. On the left is the input: a single phone photo of a frog figurine, with no turntable, scan rig, or multi-view capture. On the right is the mesh the lab machine reconstructed from that one image, checked as watertight and exported as STL. The green frog in Figure 3 is a print of this same kind of output, so the loop from photograph to physical object now runs end to end through the browser.

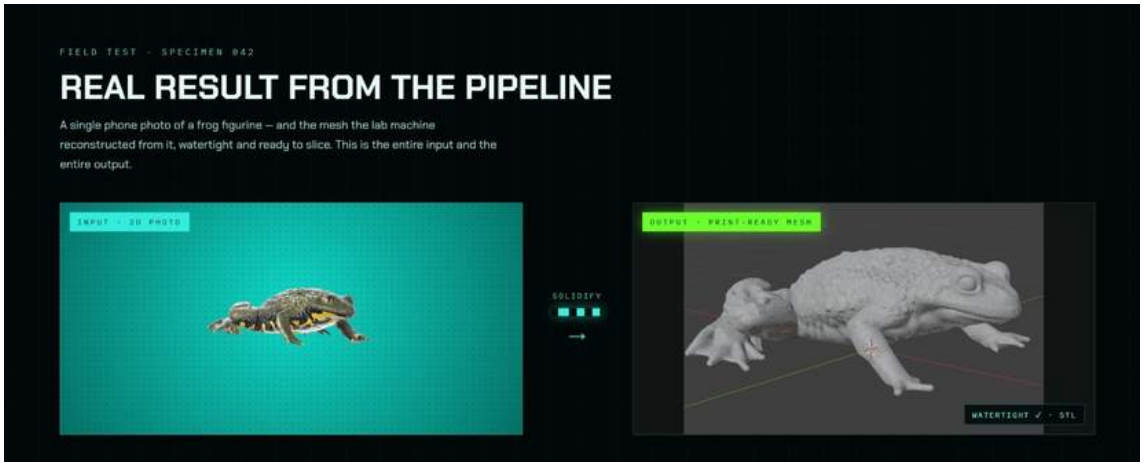

Figure 4 The deployed SOLIDIFY front end showing a single field test: the input 2D photograph of a frog figurine (left) and the print-ready mesh reconstructed from it by the lab machine (right), flagged as watertight and exported as STL. Screenshot taken from the running service.

5.3 Hosting

The service runs on the lab Mac mini and is reached over the lab's private network rather than the public internet. This keeps the GPU and the model weights inside the lab while still letting any machine in the FabLab open the front end in a browser and submit a photo.

6 Extending the Post-Processing Layer

6.1 Post-Processing Libraries and manifold3d

Having the validation layer in place, I spent time this period trying to make it better. I tried many post-processing libraries and methods to improve the quality and printability of the model's output, adding different options for post-processing so that they could be compared on the same meshes. Of everything I tried, manifold3d [7] was the best for 3D printing, so I added it to the pipeline. I kept the original trimesh pipeline as well rather than replacing it, so both paths remain available.

6.2 Reducing Non-Manifold Edges

I also did further research on the best way to decrease the number of non-manifold edges in the models, since non-manifold edges are one of the defects that make a raw mesh awkward to slice. As part of that reading I went through the TriFlow paper on generating artist-like 3D models [8], which approaches mesh quality from the generation side rather than the repair side.

That distinction is what motivated the rest of this week's work. Post-processing can only repair what the model produces; it cannot change what the model tends to produce in the first place. The next two sections cover two attempts at improving the output at the source.

7 Preference Optimization with DreamDPO

The first attempt was to integrate DreamDPO [9] concepts into the pipeline. I read the DreamDPO work on preference optimization for diffusion models, which builds pairs of candidates during generation, has a ranking model say which one is preferred, and uses that preference to steer the optimization.

Rather than using a VLM as the ranking model, I simulated the preference discriminator as a score. This did not work. The results showed no gains, and the discriminator always chose the reference path for the flow model, so the preference signal never actually moved generation away from what the model would have produced anyway. Combined with my lack of expertise in this area of the theory, I set this approach aside rather than continuing to tune it.

8 Fine-Tuning the SLat Flow Model with LoRA

The second attempt was to fine-tune the SLat flow model directly, using LoRA methods, so that the improvement is in the model weights rather than in a steering step at generation time.

8.1 Dataset

I tried this on a small sample size of 300 curated 3D models from the Thingi10K dataset [10]. Thingi10K is a collection of real 3D-printing models from Thingiverse, and curation mattered: a large part of the collection has mesh defects of exactly the kind the fine-tuning is meant to reduce, so the 300 models were filtered to clean, closed, manifold meshes before being encoded into latents for training.

8.2 Training and Results

Initial training showed gains. The clearest result was on non-manifold edges: the fine-tuned model produced significantly less non-manifold edges for raw results - that is, measured on the model's direct output, before manifold3d or the trimesh pipeline runs on it - and this came without loss in detail.

Measured on raw output	Base model	Fine-tuned
Non-manifold edge rate	1.68%	**0.31%**
Open edge rate	0.96%	**0.51%**
Separate components	2,350	**447**
Detail	no loss - slightly higher	

Because these gains are on the raw output, they are made before manifold3d and the trimesh pipeline run, not instead of them. Both stay in the pipeline. I am continuing to collect more latent data for training, since 300 models is a small sample and the amount of training data is the main limit on how far this can go.

9 Discussion

The main practical issue with the model has been that its raw output is not directly printable. Building the validation/post-processing layer described in Section 4 fixed that, and deploying the pipeline as a web service has changed what the remaining problems are. With the front end in place, the bottleneck is no longer whether the mesh is printable but the practical side of running a service: how long a single generation takes, and what happens when more than one FabLab user submits a photo at the same time. The applied work over this period has therefore been the validation layer and the deployment, on top of the background theory covered in Section 3.

10 Limitations and Next Steps

The validation layer is still new and has not been tested on many samples, so its results should be treated as preliminary. The web service is newer still: it runs on a single machine over the lab network and has not been tested under concurrent use, and the field test in Figure 4 is one specimen rather than a systematic evaluation. The fine-tuning result is also preliminary - it was trained on 300 models, which is a small sample. Next, I plan to put the service in front of actual FabLab users and collect failure cases, handle queued requests so that simultaneous submissions do not collide, keep testing the validation layer on more model outputs, collect more latent data for fine-tuning, and continue working through the flow and diffusion theory where the lecture notes currently leave off.

11 Conclusion

Over the past few weeks I reviewed the background theory behind TRELLIS: the TRELLIS pipeline itself and the flow matching, guidance, VAE, and transformer-architecture material from the MIT lecture notes. I also ran the model and built a validation layer to post-process its output into a printable form, with physical prints included above, and put that pipeline behind a web front end, SOLIDIFY, hosted on the lab machine, so that a FabLab user can upload a single photograph and get back a watertight, print-ready mesh. This week I extended the post-processing side with manifold3d, tried and set aside a DreamDPO-based preference approach, and fine-tuned the SLat flow model with LoRA on 300 curated Thingi10K models, which gave significantly less non-manifold edges on the raw output without loss in detail. Collecting more latent data for training, and testing the service with real users, is the next step.

References

1. P. Holderrieth and E. Erives. Introduction to Flow Matching and Diffusion Models. MIT 6.S184 course notes and lectures, 2026. https://diffusion.csail.mit.edu/2026/
2. J. Xiang, Z. Lv, S. Xu, Y. Deng, R. Wang, B. Zhang, D. Chen, X. Tong, and J. Yang. Structured 3D Latents for Scalable and Versatile 3D Generation (TRELLIS). arXiv:2412.01506, 2024.
3. J. Xiang, X. Chen, S. Xu, R. Wang, Z. Lv, Y. Deng, H. Zhu, Y. Dong, H. Zhao, N. J. Yuan, and J. Yang. Native and Compact Structured Latents for 3D Generation (TRELLIS-2). arXiv:2512.14692, 2025.
4. O. Simeoni et al. DINOv3. arXiv:2508.10104, 2025.
5. Interactive explainer: How TRELLIS-2 Works. https://claude.ai/public/artifacts/c52594ab-8c5c-404c-8999-68e62b4054ee
6. SOLIDIFY web service (internal, lab network). https://scifablabs-mac-mini.tailfc1a5e.ts.net/
7. Manifold3D - mesh library for manifold geometry. https://github.com/elalish/manifold
8. TriFlow - paper on generating artist-like 3D models.
9. Z. Zhou, X. Xia, F. Ma, H. Fan, Y. Yang, and T.-S. Chua. DreamDPO: Aligning Text-to-3D Generation with Human Preferences via Direct Preference Optimization. arXiv:2502.04370, 2025.
10. Q. Zhou and A. Jacobson. Thingi10K: A Dataset of 10,000 3D-Printing Models. arXiv:1605.04797, 2016.